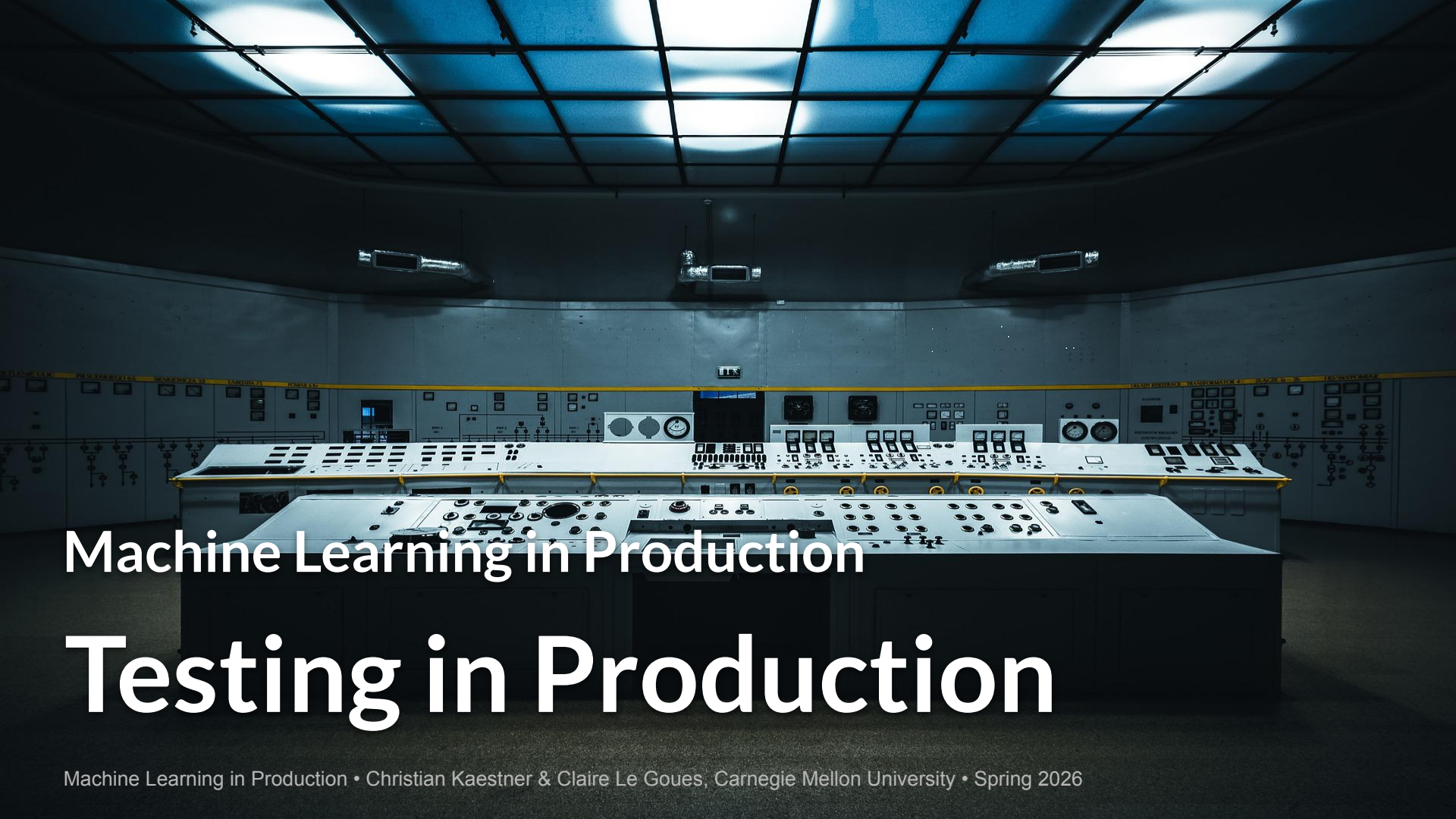




Machine Learning in Production Testing in Production

Logistics

- Project M1 due tonight, peer feedback due 24h after submitting
- Schedule debriefing meetings with your TA mentor this week
- Midterm exam next week Wednesday (Feb 25), regular time and place for lectures, see #announcements for details
 - Questions based on shared scenario, apply concepts
 - All lectures and readings in scope, focus on concepts with opportunity to practice (e.g., recitations, homeworks, in-class exercises, reading quizzes)
 - Closed book, but 6 sheets of notes (sorry, no ChatGPT)
 - No lab next week (Feb 27), enjoy Spring Break!
- Milestone M2 planned for 2 weeks but scheduled over 4 weeks due to midterm and Spring break, don't start last minute



Changelog 
@changelog · [Follow](#)



“Don’t worry, our users will notify us if there’s a problem”



10:03 AM · Jun 8, 2019



1.9K



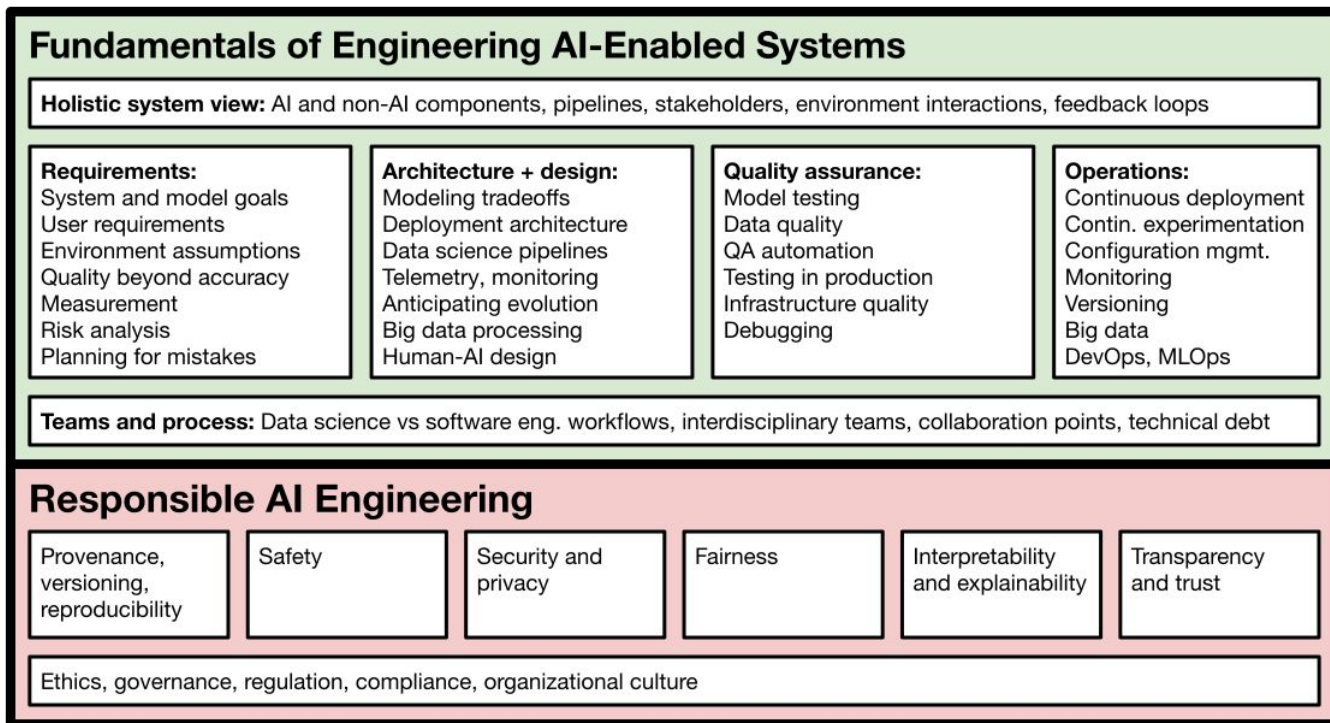
Reply



Copy link

[Read 11 replies](#)

Back to QA...



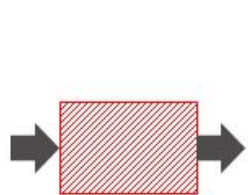
Learning Goals

- Design telemetry for evaluation in practice
- Understand the rationale for beta tests and chaos experiments
- Plan and execute experiments (chaos, A/B, shadow releases, ...) in production
- Conduct and evaluate multiple concurrent A/B tests in a system
- Perform canary releases
- Examine experimental results with statistical rigor
- Support data scientists with monitoring platforms providing insights from production data

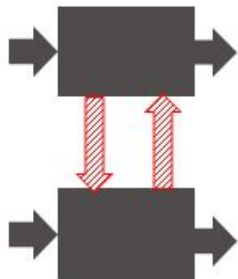
From Unit Tests to Testing in Production

Unit Test, Integration Tests, System Tests

These are all testing *before* deployment --- Manual or automated



Unit testing



Integration testing



System testing



Acceptance
testing
(Demonstration)

Beta Testing: Move towards testing during deployment

Early release to select users, asking them to send feedback or report issues.



Crash Telemetry: "Bugs" in Production

- **Telemetry:** The automated collection, transmission, and analysis of data about the performance, behavior, and usage of an application or system in real time.
- With internet availability, send crash reports home to identify problems "in production".



Model Assessment in Production

Recap: Limitations of Offline Model Eval

Training and test data drawn from the same population

- **i.i.d.: independent and identically distributed**
- leakage and overfitting problems quite common

Is the population representative of production data?

- If not or only partially or not anymore: Does the model generalize beyond training data?

Model Assessment in Production

Live observation in the running system

Need **telemetry** to evaluate quality – automated collection and monitoring of data on model performance, data quality, and user behavior to ensure continuous model accuracy, detect drift, and maintain system health in real-time.

How to collect data?

How do you gather data without being intrusive (i.e., labeling outcomes), without harming user experience?

- Was the house price predicted correctly?
- Was the ranking of search results good?
- Was the meeting summary good?
- Was there cancer in the image?
- Was a Spotify playlist good?
- Was the weather prediction good?
- Was the translation correct?
- Did the self-driving car break at the right moment? Did it detect the pedestrians?



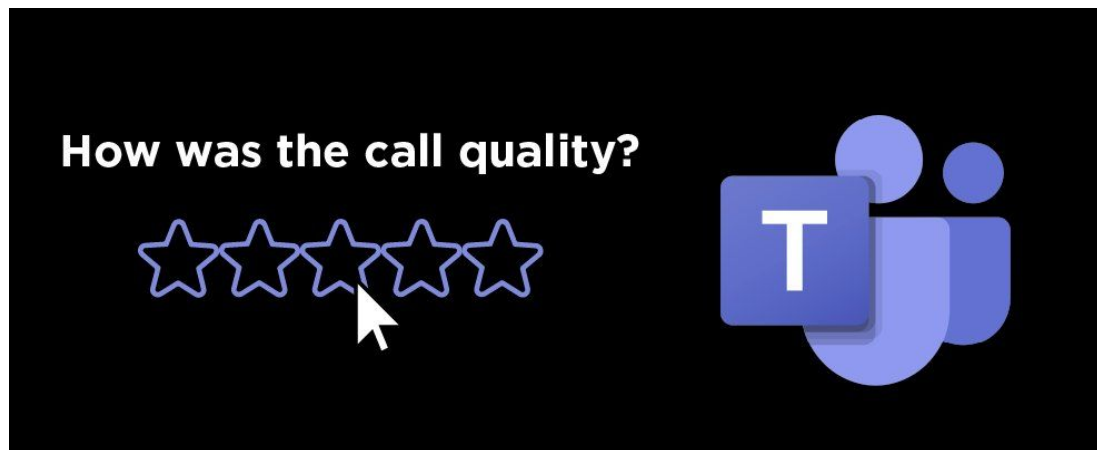
Telemetry strategy: “Manually” Label Production Samples

Akin to labeling learning and testing data, have human/AI annotators



scale
Labelbox

Telemetry strategy: Explicit Report



 Report a problem `Ctrl+Alt+Shift+R`

 Give a compliment

 Suggest a feature

Expect only sparse feedback and expect negative feedback over-proportionally

Telemetry strategy: Implicit Feedback

Collect user editing ("human-AI collaboration"); Can have indicators for low confidence predictions

The screenshot shows a web interface for a transcription service. At the top, it says "the-changelog-318" and "Quality: High". Below this is a progress bar and a "Share" button. The main area displays a transcript from "Speaker 5" starting at 07:44. The transcript text is: "Yeah. So there's a slight story behind that. So back when I was in, uh, Undergrad, I wrote a program for myself to measure a, the amount of time I did data entry from my father's business and I was on windows at the time and there wasn't a function called time dot [inaudible] time, uh, which I needed to parse dates to get back to time, top of representation, uh, I figured out a way to do it and I gave it to what's called the python cookbook because it just seemed like something other people could use. So it was just trying to be helpful. Uh, subsequently I had to figure out how to make it work because I didn't really have to. Basically, it bothered me that you had to input all the locale information and I figured out how to do it over the subsequent months. And actually as a graduation gift from my Undergrad, the week following, I solved it and wrote it all out." The transcript includes several highlighted corrections in orange: "a, the", "from", "[inaudible]", "it was", "because", "the", and "it". At the bottom of the transcript, there is a feedback prompt: "How did we do on your transcript?" followed by five empty star icons.

the-changelog-318
← Dashboard | Quality: High ⓘ
Last saved a few seconds ago ... Share

00:00 Offset 00:00 01:31:27

Play Back 5s 1x Speed Volume

NOTES
Write your notes here

Speaker 5 ▶ 07:44

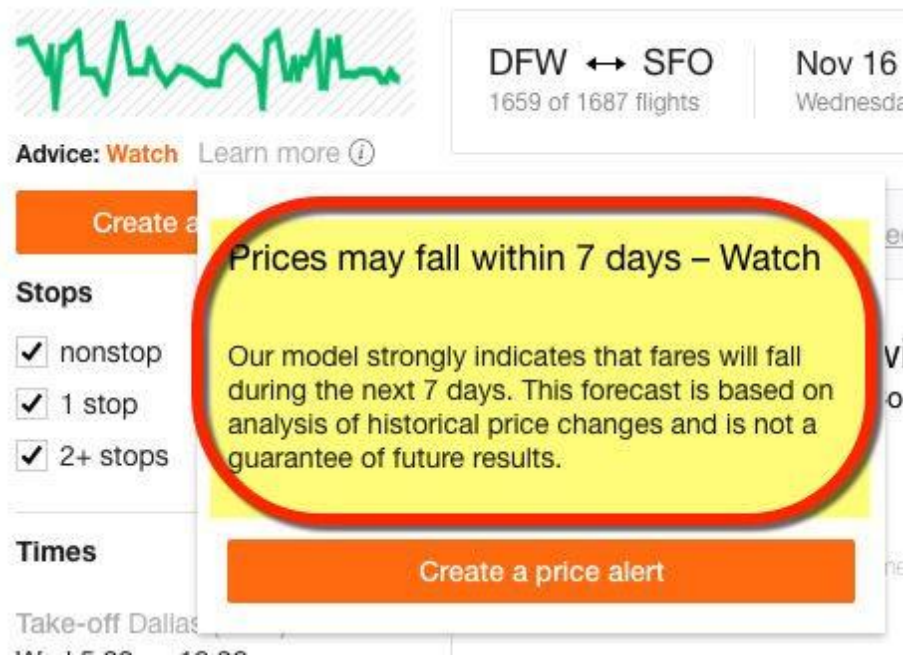
Yeah. So there's a slight story behind that. So back when I was in, uh, Undergrad, I wrote a program for myself to measure a, the amount of time I did data entry from my father's business and I was on windows at the time and there wasn't a function called time dot [inaudible] time, uh, which I needed to parse dates to get back to time, top of representation, uh, I figured out a way to do it and I gave it to what's called the python cookbook because it just seemed like something other people could use. So it was just trying to be helpful. Uh, subsequently I had to figure out how to make it work because I didn't really have to. Basically, it bothered me that you had to input all the locale information and I figured out how to do it over the subsequent months. And actually as a graduation gift from my Undergrad, the week following, I solved it and wrote it all out.

Speaker 5 ▶ 08:38

And I asked, uh, Alex Martelli, the editor of the Python Cookbook, which had published my original recipe, a, how do I get this into python? I think it might help

How did we do on your transcript? ☆☆☆☆☆

Telemetry strategy: Wait-and-See (Retrospective)



The screenshot displays a flight search interface. At the top left, a green line graph shows price fluctuations. To its right, the route 'DFW ↔ SFO' is specified, along with the date 'Nov 16 Wednesday' and the number of flights '1659 of 1687 flights'. Below this, an 'Advice: Watch' notification is shown with a 'Learn more' link. A yellow alert box with a red border is prominently displayed in the center, containing the text: 'Prices may fall within 7 days – Watch' and 'Our model strongly indicates that fares will fall during the next 7 days. This forecast is based on analysis of historical price changes and is not a guarantee of future results.' Below the alert box is an orange button labeled 'Create a price alert'. On the left side of the interface, there are sections for 'Stops' (with checkboxes for nonstop, 1 stop, and 2+ stops) and 'Times' (with a 'Take-off Dallas' option).

DFW ↔ SFO
1659 of 1687 flights

Nov 16
Wednesday

Advice: **Watch** [Learn more](#)

Create a price alert

Stops

- ☒ nonstop
- ☒ 1 stop
- ☒ 2+ stops

Times

Take-off Dallas

Prices may fall within 7 days – Watch

Our model strongly indicates that fares will fall during the next 7 days. This forecast is based on analysis of historical price changes and is not a guarantee of future results.

Create a price alert

Summary: Telemetry Strategies

Ordered by how much efforts needed:

- Manual/crowdsourced labeling on production samples
- Explicit report: Ask users or allow users to complain
- Observe user reaction (implicit feedback)
- Wait and see

Measuring Model Quality with Telemetry

Three steps, again on measurements: (1) Metric, (2) Data collection (telemetry), (3) Operationalization

- Telemetry can provide insights for correctness
 - sometimes very accurate labels for real unseen data
 - sometimes only mistakes
 - sometimes delayed
 - often just samples
 - often just weak proxies for correctness
- Often sufficient to *approximate* precision/recall or other model-quality measures
- Mismatch to (static) eval set may indicate stale or unrepresentative data
- Trend analysis can provide insights even for inaccurate proxy measures

Breakout: Design Telemetry in Production

Discuss how to collect telemetry (manual/crowdsourced labeling, wait and see, ask users, observe user reaction), the metric to monitor, and how to operationalize.

Scenarios:

- Front-left: Amazon: Shopping app detects the shoe brand from photos
- Front-right: Google: Tagging uploaded photos with friends' names
- Back-left: Spotify: Recommended personalized playlists
- Back-right: Zoom: automatic meeting summary

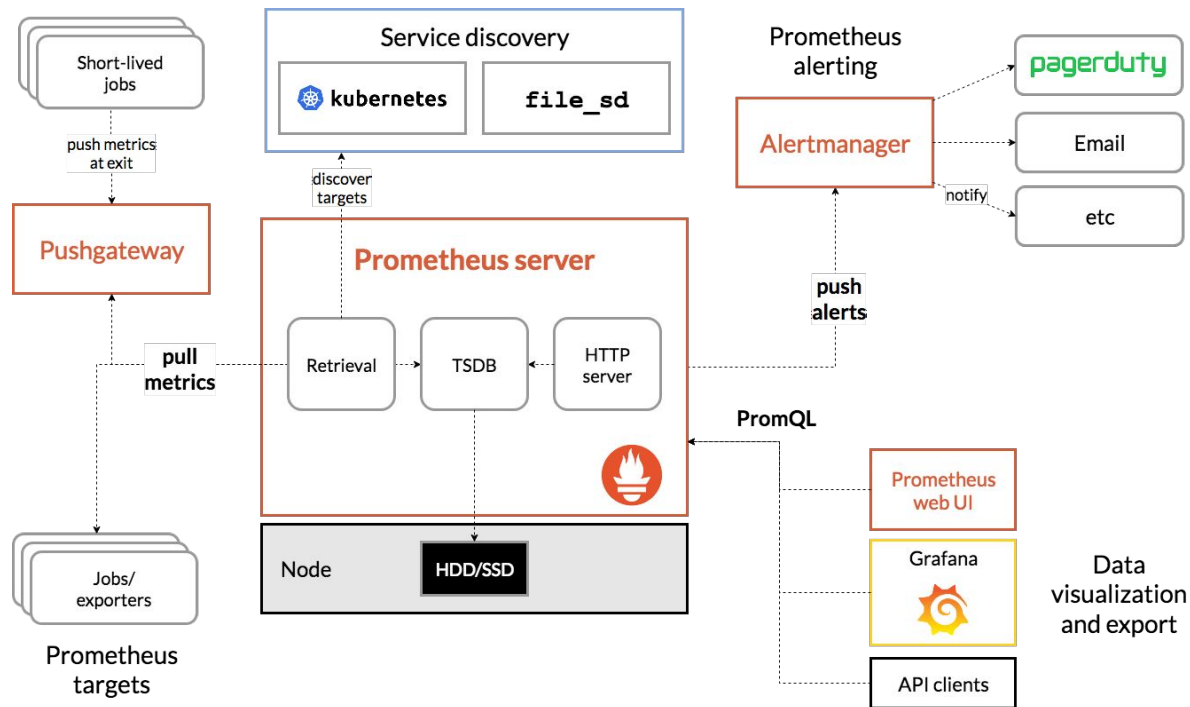
As a group post to #1lecture and tag team members:

- Model quality metric:
- Data to collect:
- Operationalization:

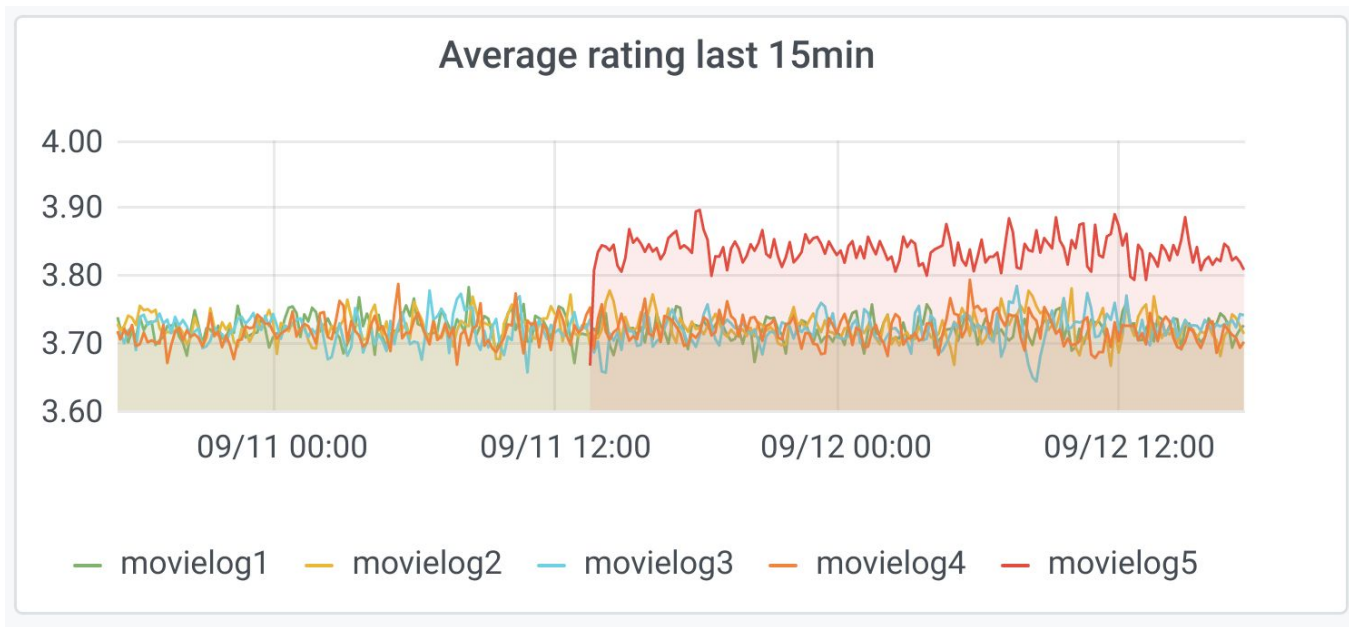
Monitoring Model Quality in Production

- Monitor model quality together with other quality attributes (e.g., uptime, response time, load)
- Set up automatic alerts ...
 - When model quality drops
 - Watch for jumps after releases (roll back after negative jump)
 - Watch for slow degradation (Stale models, data drift, feedback loops, adversaries)
- Debug common or important problems
 - Monitor characteristics of requests
 - Mistakes uniform across populations?
 - Challenging problems → refine training, add regression tests

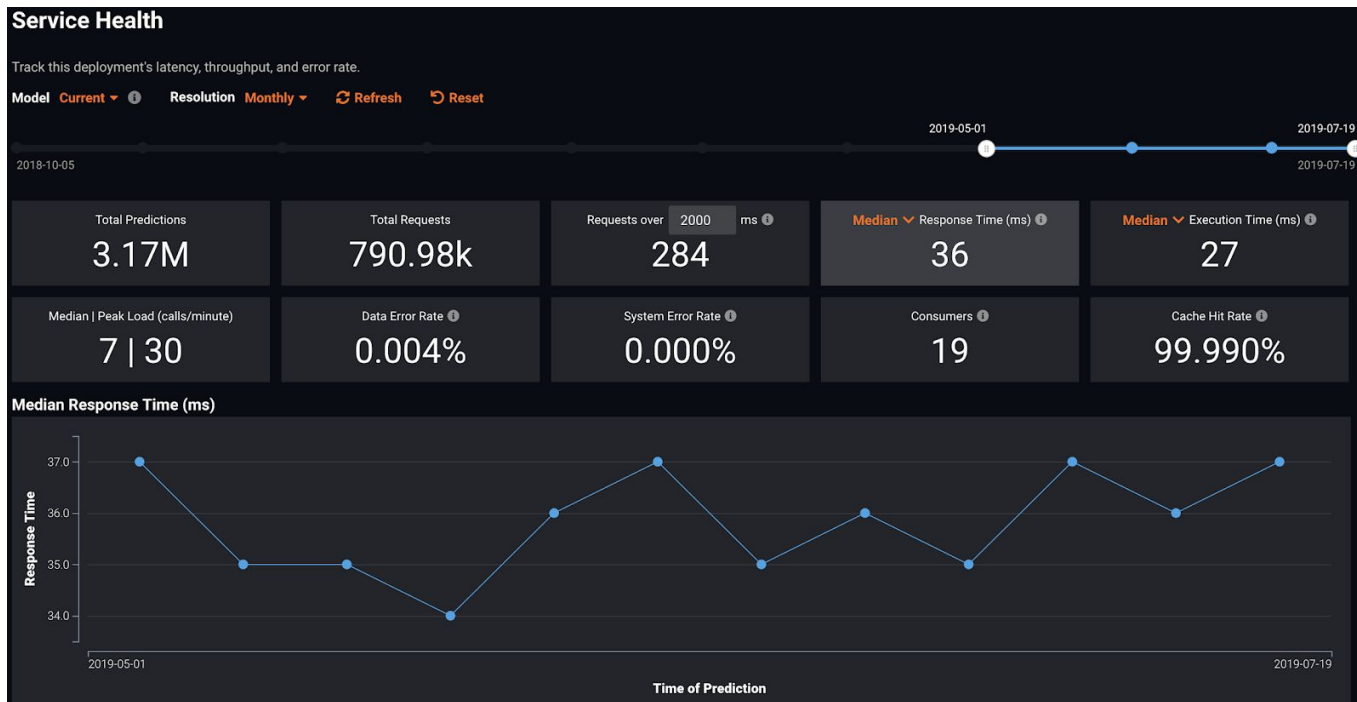
Monitoring services: Prometheus and Grafana



Grafana screenshot from Movie Recommendation Service



Many commercial solutions



Many different monitoring possibilities, e.g. Detecting Drift

Deploy monitoring at ...

Training data

- Schema & distribution of incoming data
- Distribution of labels

Requests & predictions

- Schema & distribution of requests
- Distribution of predictions
- Quality of predictions

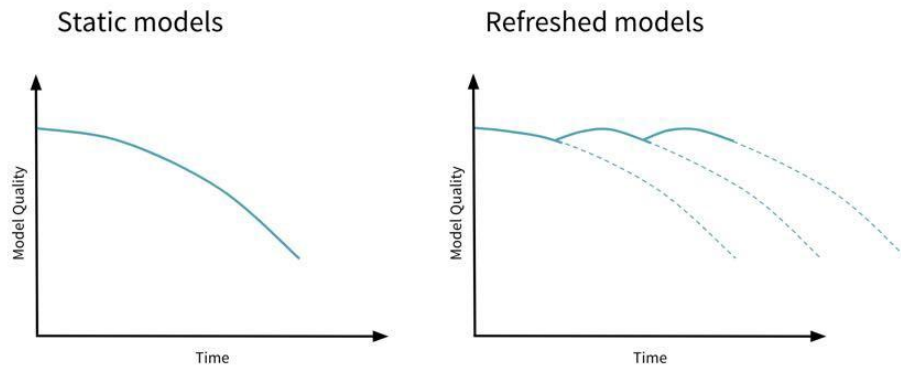
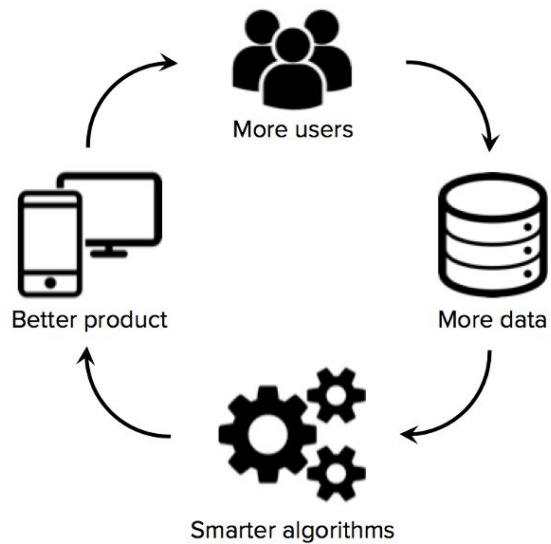


Image source: Joel Thomas and Clemens Mewald. [Productionizing Machine Learning: From Deployment to Drift Detection](#). Databricks Blog, 2019

Monitoring without Ground Truth: Invariants/Assertions to Assure with Telemetry

- Consistency between multiple sources
 - e.g., multiple models agree, multiple sensors agree
 - e.g., text and image agree
- Physical domain knowledge
 - e.g., cars in video shall not flicker,
 - e.g., earthquakes should appear in sensors grouped by geography
- Domain knowledge about unlikely events
 - e.g., unlikely to have 3 cars in same location
- Stability
 - e.g., object detection should not change with video noise
- Input conforms to schema (e.g. boolean features)
- And all invariants from model quality lecture, including capabilities

Telemetry for Training: The ML Flywheel



graphic by [CBInsights](#)

Engineering Challenges for Telemetry

Data volume and operating cost

- e.g., record "all AR live translations"?
- reduce data through sampling
- reduce data through summarization (e.g., extracted features rather than raw data; extraction client vs server side)

Isolating feedback for specific ML component + version

Biased sampling / Rare events

Privacy (related: Offline deployments)



Breakout: Engineering Challenges in Telemetry

Discuss: **Cost, privacy, rare events, bias**

Scenarios:

- Front-left: Amazon: Shopping app feature that detects the shoe brand from photos
- Front-right: Google: Tagging uploaded photos with friends' names
- Back-left: Spotify: Recommended personalized playlists
- Back-right: Zoom: automatic meeting summary

(can update Slack, but not needed)

Revisiting Model Quality vs System Goals

Model Quality vs System Goals

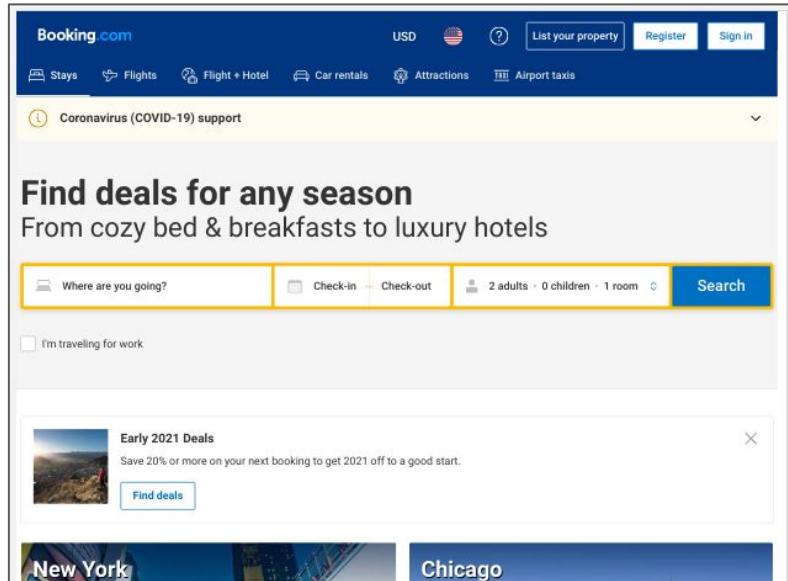
Telemetry can approximate model accuracy

Telemetry can directly measure system qualities, leading indicators, user outcomes

- define measures for "key performance indicators"
- clicks, buys, signups, engagement time, ratings
- operationalize with telemetry

Model Quality vs System Quality: booking.com

What's the system goal?



The screenshot displays the Booking.com website interface. At the top, there is a dark blue header with the 'Booking.com' logo, currency set to 'USD', and links for 'List your property', 'Register', and 'Sign in'. Below the header, a navigation bar includes icons and labels for 'Stays', 'Flights', 'Flight + Hotel', 'Car rentals', 'Attractions', and 'Airport taxis'. A yellow banner for 'Coronavirus (COVID-19) support' is visible. The main section features the heading 'Find deals for any season' with the subtext 'From cozy bed & breakfasts to luxury hotels'. The search bar contains the text 'Where are you going?' and 'Check-in + Check-out', with a dropdown menu showing '2 adults · 0 children · 1 room'. A blue 'Search' button is to the right. Below the search bar, there is a checkbox for 'I'm traveling for work'. A promotional banner for 'Early 2021 Deals' offers a 20% discount on the next booking. At the bottom, there are two large blue buttons for 'New York' and 'Chicago'.

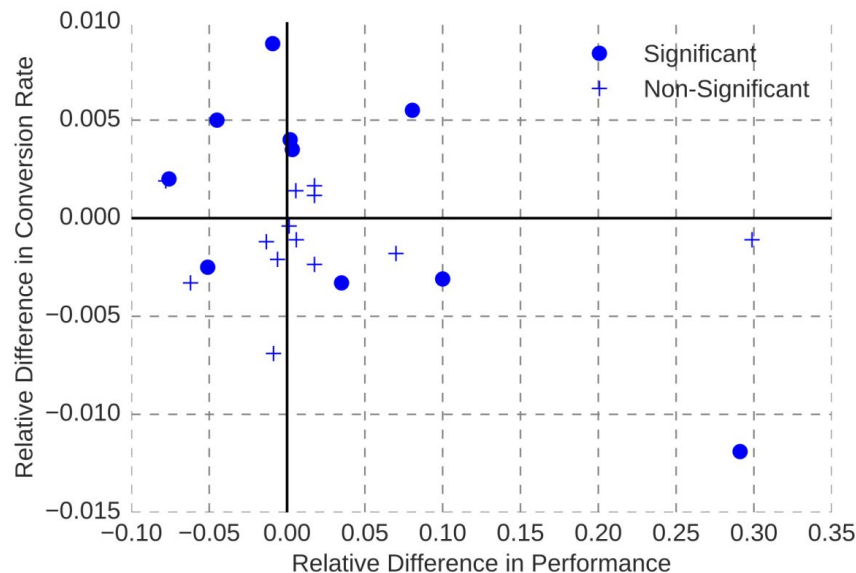
Bernardi, Lucas, et al. "150 successful machine learning models: 6 lessons learned at Booking.com." In Proc. Int'l Conf. Knowledge Discovery & Data Mining, 2019.

Possible causes of model vs system conflict?

Observed no correlation between offline performance gain vs. business value gain, across many different models with different goals.

Possible reasons:

- Value Performance Saturation
- Uncanny Valley effect (e.g., "creepy AIs")
- Proxy Over-optimization (e.g., *click-through rate approximating conversion*)



Bernardi, Lucas, et al. "150 successful machine learning models: 6 lessons learned at Booking.com." In Proc. Int'l Conf. Knowledge Discovery & Data Mining, 2019.

Design Telemetry in Production

Discuss: **What key performance indicator of the *system* to collect?**

Scenarios:

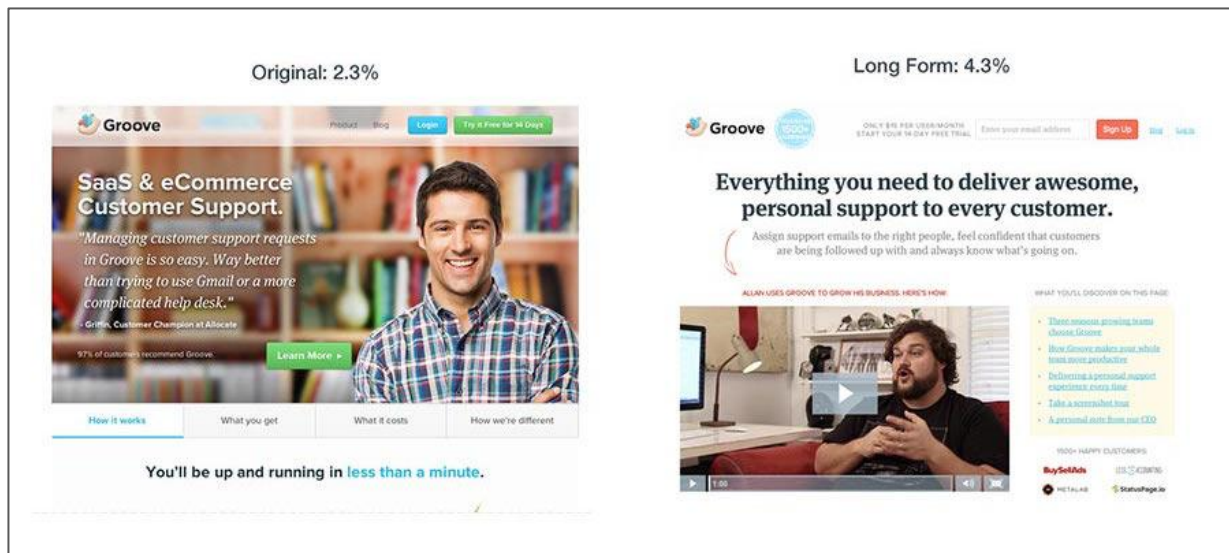
- Front-left: Amazon: Shopping app feature that detects the shoe brand from photos
- Front-right: Google: Tagging uploaded photos with friends' names
- Back-left: Spotify: Recommended personalized playlists
- Back-right: Zoom: automatic meeting summary

Experimenting in Production

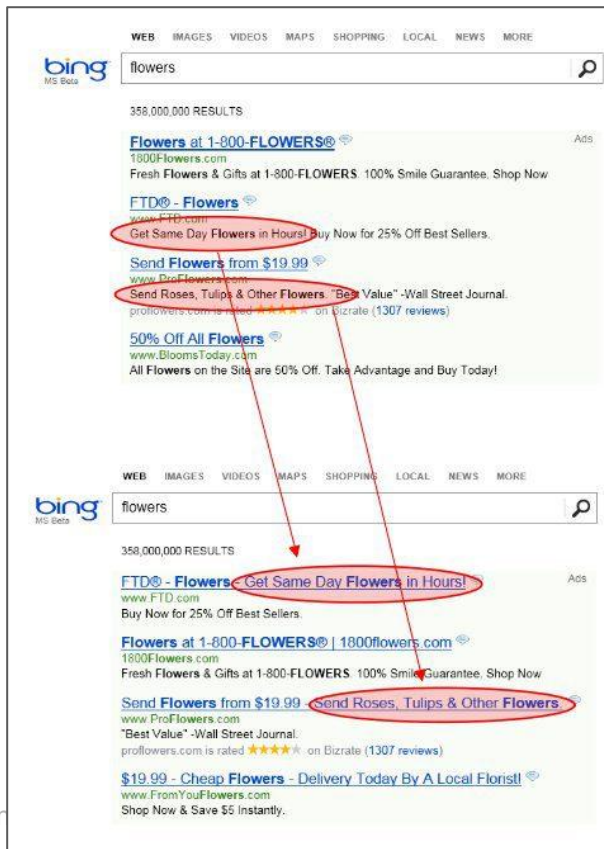
- A/B experiments
- Shadow releases / traffic teeing
- Blue/green deployment
- Canary releases
- Chaos experiments

A/B Experiments: What if...?

- ... we have plenty of subjects for experiments
- ... we could randomly assign to treatment/ control group without them knowing
- ... we could analyze small individual changes and keep everything else constant



Bing Experiment



- Experiment: Ad display at Bing
- Suggestion deprioritized and left in the backlog for over 6 months
- Implemented & A/B tested in production
- Within 2h *revenue-too-high* alarm triggered.
 - Typically suggest serious bug (e.g., double billing)
 - But valid here: Revenue increase by 12% - \$100M annually in US
 - Did not hurt user-experience metrics
- Simple adjustment became one of the most profitable ideas in Bing's history!

From: Kohavi, Ron, Diane Tang, and Ya Xu. "[Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing](#)." 2020.

A/B Experiment for ML Components?

- New product recommendation algorithm for web store?
- New language model in audio transcription service?
- New (offline) model to detect falls on smart watch?



Implementing A/B Testing

Implement alternative versions of the system

- using feature flags (decisions in implementation)
- separate deployments (decision in router/load balancer)

Map users to treatment group

- Randomly from distribution
- Static user - group mapping
- Online service (e.g., [launchdarkly](#), [split](#))

Monitor outcomes *per group*

- Telemetry, sales, time on site, server load, crash rate

Feature Flags (Boolean flags)

```
if (features.enabled(userId, "one_click_checkout")) {  
    // new one click checkout function  
} else {  
    // old checkout functionality  
}
```

- Good practices: tracked explicitly, documented, keep them localized and independent
- External mapping of flags to customers, who should see what configuration
 - e.g., 1% of users sees one_click_checkout, but always the same users; or 50% of beta-users and 90% of developers and 0.1% of all users

```
def isEnabled(user): Boolean = (hash(user.id) % 100) < 10
```


Feature Flags as a Service

▼ Treatments ⓘ | 2 treatments, if Split is killed serve the default treatment of "off"

Treatment	Default	Description
on		The new version of registration process is enabled.
off		The old version of registration process is enabled.

+ Add treatment | [Learn more about multivariate treatments.](#)

▼ Whitelist ⓘ | 0 user(s) or segments individually targeted.

+ Add whitelist

▼ Traffic Allocation ⓘ | 100% of user included in Split rules evaluation below.

Total Traffic Allocation: 100 % total User in Split

▼ Targeting Rules ⓘ | 2 rules created for targeting.

if user is in segment qa Then serve  on

else if user is in segment beta_testers Then serve percentage

 on	50
 off	50

+ Add rule

▼ Default Rule ⓘ | Serve treatment of "off".

serve  off

Confidence in A/B Experiments (statistical tests)

Group A

*classic personalized content
recommendation model*

2158 Users

average 3:13 min time on site

Group B

*updated personalized content
recommendation model*

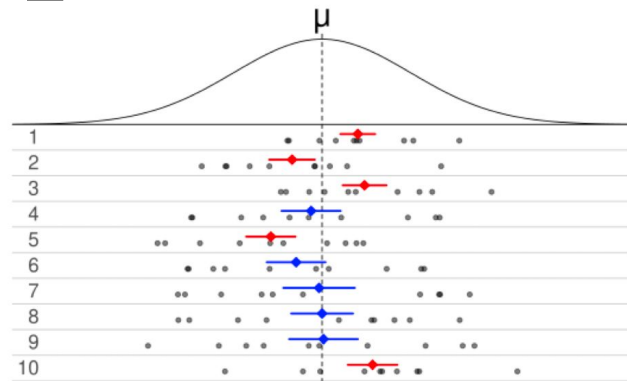
10 Users

average 3:24 min time on site

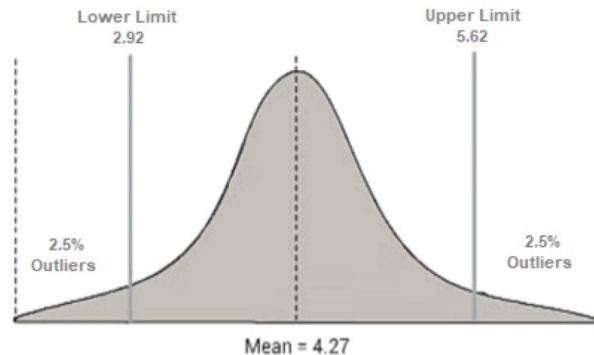
What's the problem of comparing the average?

Analyzing Results: Stats 101

- All the data have some **distribution**.
- **Normal distribution** is very common (e.g. students tend to get average score, not full score).
- When we draw samples from the same distribution, we can get different samples **just by chance**.
- Statistics tells us whether “interesting” patterns we observe **actually exist** in the samples, or whether it’s just sampling noise — use some numbers to express uncertainty.
- **Confidence interval**: if we can compute the average of some samples (average on-site time of 10 users), what’s the average of the population (whole user group)?



95% Chance your population mean will fall between 2.92 and 5.62

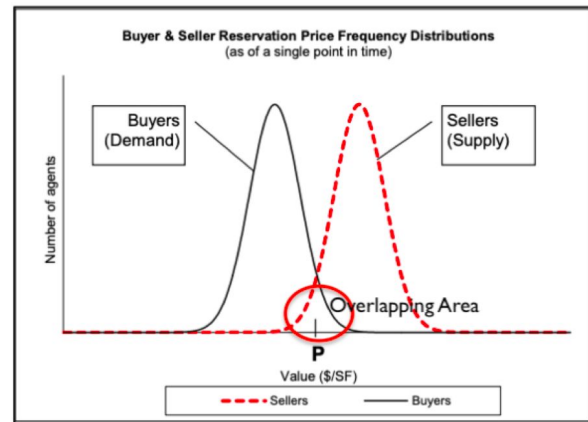
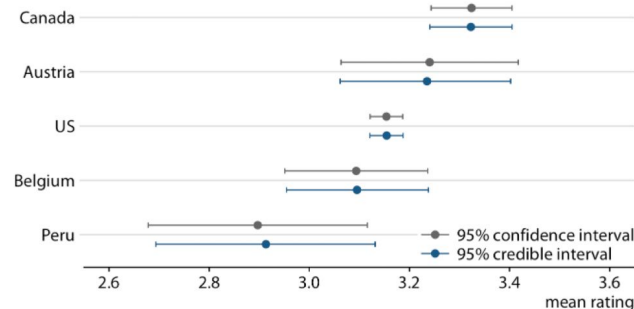


Analyzing Results: Stats 101

Significance testing also helps with comparison. When most of the confidence intervals overlap, we'd know there's no actual differences between groups.

This is quantified by **p-value**:

- Misconception: “what is the probability a difference between two samples is by chance?” (lower = higher likelihood of a sig. difference)
- Reality: the p-value starts with the assumption that there's no real difference (null hypothesis), then asks "how likely would our observed data be in that scenario?"



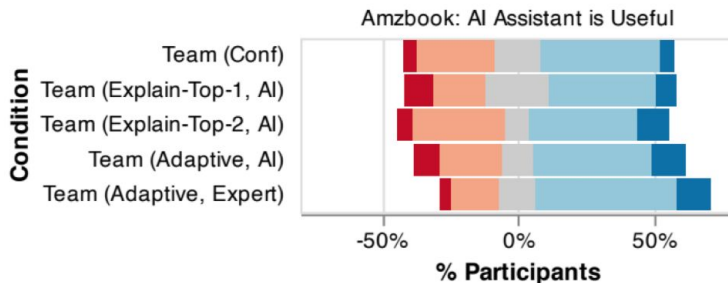
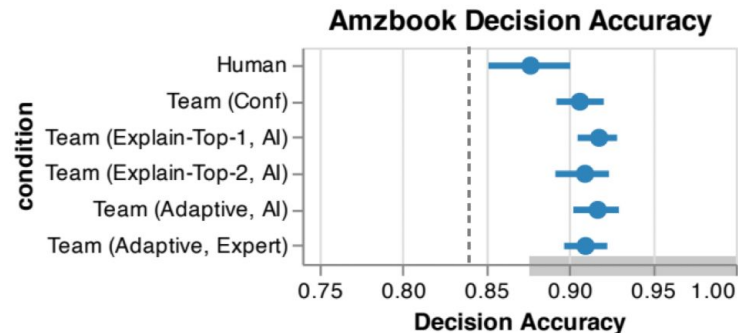
Stats 101: How to compute p-value?

Parametric tests: Assume comparing normally distributed groups that have the same variances.

- Tests: t-test, ANOVA, & linear regression.
- For: model accuracy, human click streams
- More sensitive and powerful when the requirement is met.

Non-parametric tests: Does not assume normal distribution.

- For: categorical data like Likert Scale rating
- Tests: Wilcoxon signed-rank test compare users' nominal Likert Scale ratings



t-test

We will ask for statistical test in M3 – Many libraries implement it!

```
> t.test(x, y, conf.level=0.9)

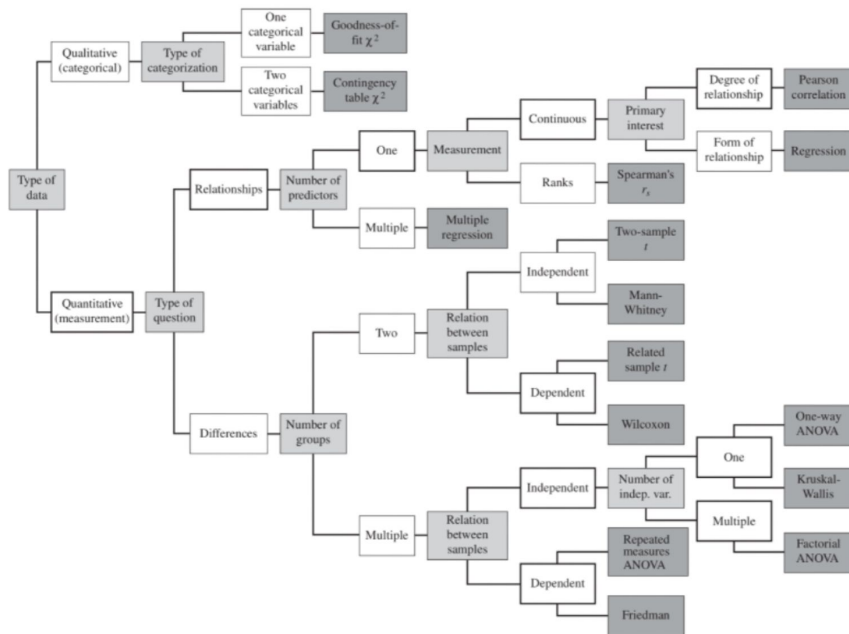
Welch Two Sample t-test

t = 1.9988, df = 95.801, p-value = 0.04846
alternative hypothesis: true difference in means is
not equal to 0
90 percent confidence interval:
 0.3464147 3.7520619
sample estimates:
mean of x mean of y
 51.42307  49.37383

> # paired t-test:
> t.test(x-y, conf.level=0.9)
```

Decision tree of tests

Many many other factors, e.g., dependent vs. independent measures



How many samples needed?

Too few?

Noise and random results!

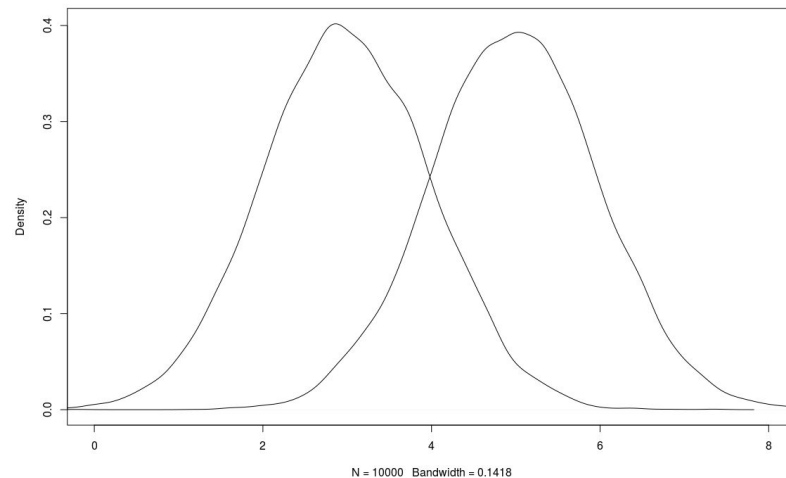
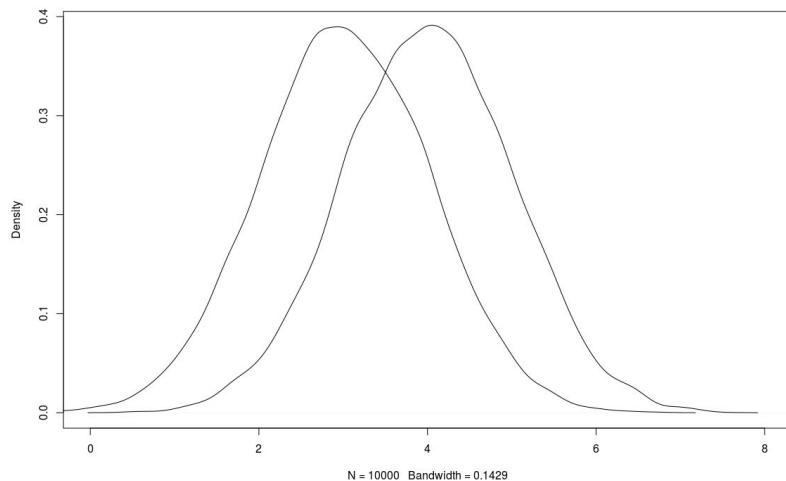
Too many?

Risk of spreading bad designs!

Some factors that affect your sample size

Different effect size, same deviations

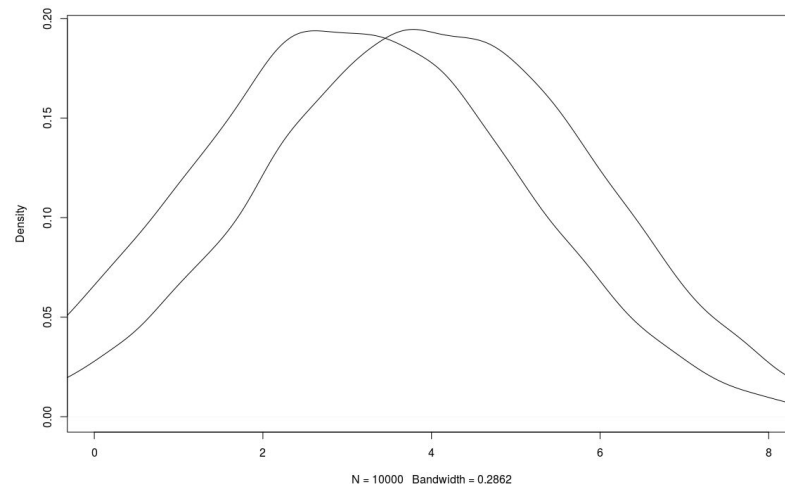
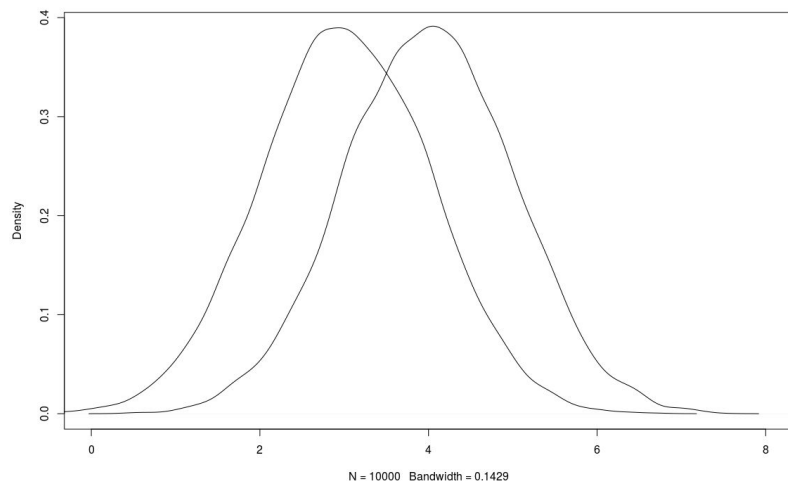
- Effect size: the magnitude of the difference between groups



Some factors that affect your sample size

Same effect size, different deviations

- Less noise --> Easier to recognize



Concurrent A/B testing

Multiple experiments at the same time

- Independent experiments on different populations – interactions not explored
- Multi-factorial designs, well understood but typically too complex, e.g., not all combinations valid or interesting
- Grouping in sets of experiments (layers)

Other Experiments in Production

Chaos experiments

Shadow releases / traffic teeing

Canary releases

Chaos Experiments: deliberate faults in production to test robustness



Chaos Experiments for ML Components?



Shadow releases / traffic teeing

Run both models in parallel

Use predictions of old model in production

Compare differences between model predictions

If possible, compare against ground truth labels/telemetry

Examples?

Canary Releases

Release new version to small percentage of population (like A/B testing)

Automatically roll back if quality measures degrade

Automatically and incrementally increase deployment to 100% otherwise



Advice for Experimenting in Production

Minimize *blast radius* (canary, A/B, chaos expr)

Automate experiments and deployments

Allow for quick rollback of poor models (continuous delivery, containers, load balancers, versioning)

Make decisions with confidence, compare distributions

Monitor, monitor, monitor

Summary

Production data is ultimate unseen validation data

Both for model quality and system quality

Telemetry is key and challenging (design problem and opportunity)

Monitoring and dashboards

Many forms of experimentation and release (A/B testing, shadow releases, canary releases, chaos experiments, ...) to minimize "blast radius"; gain confidence in results with statistical tests

Further Readings

- On canary releases: Alec Warner and Štěpán Davidovič. “[Canary Releases](#).” in [The Site Reliability Workbook](#), O’Reilly 2018
- Everything on A/B testing: Kohavi, Ron. [Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing](#). Cambridge University Press, 2020.
- A/B testing critiques: Josh Constine. [The Morality Of A/B Testing](#). Blog 2014; the [Center of Humane Technology](#); and the Netflix documentary [The Social Dilemma](#)
- Jens Meinicke, Chu-Pan Wong, Bogdan Vasilescu, and Christian Kästner. [Exploring Differences and Commonalities between Feature Flags and Configuration Options](#). In Proceedings of the Proc. International Conference on Software Engineering ICSE-SEIP, pages 233–242, May 2020.
- LLM validation: Shankar et al. [Who Validates the Validators? Aligning LLM-Assisted Evaluation of LLM Outputs with Human Preferences](#). In Proc of ACM Symposium on User Interface Software and Technology (UIST '24), 2024.